Website Auditor & Dashboard — Developer Handover Report

Repository: natasha-a-a/auditor · Prepared for developer handover

1. Overview

This repository contains **two separate Streamlit applications** that share a common data store and utility modules:

- **The Auditor** (auditor_app.py, ~1,440 lines) — the **write** side. It crawls websites, runs seven audit categories, scores each site, and persists results.
- **The Dashboard** (dashboard.py, ~580 lines) — the **read** side. It consumes stored audit results for analysis, statistics, trend charts, and lead-generation CSV exports.

They communicate **exclusively through CSV files stored in GitHub** (plus the shared config CSVs). There is no database, no API between them, and no shared code beyond two small utility modules. Both apps are standalone Streamlit entry points (`streamlit run auditor_app.py` / `streamlit run dashboard.py`).

Supporting modules (shared by both apps)

- **html_parser.py** (151 lines) — CachedSoup, a wrapper around a BeautifulSoup object providing cached lowercase text extraction (text_lower) and helper checks: has_viewport(), check_contact_form(), has_analytics(), plus delegated find/find_all via __getattr__.
- **github_cache.py** (73 lines) — in-memory CSV fetch cache (get_github_csv(url)) with a configurable TTL (default 2 hours; the dashboard sets it to 1 hour). Includes force_refresh and sep parameters for the semicolon-delimited recommendations CSV.

Configuration/data files (CSV-driven; both apps read these)

- **thresholds.csv** — tunable limits: LOAD_TIME_THRESHOLD=3.0, MAX_URL_DEPTH=3, MIN_INTERNAL_LINKS=5, COPYRIGHT_YEAR_THRESHOLD=2, CACHE_CLEANUP_DAYS=365, BATCH_SIZE=50, WHOIS_RETRIES=2.
- **score_deductions.csv** — point weights per failed check (e.g., ssl_invalid=30, domain_expiring=25, no_products=20).
- **industry_benchmarks.csv** — per-industry, per-category target scores.
- **competitors.csv** — per-industry list of competitor URLs to auto-crawl for benchmarking (currently near-empty: header only).
- **language_keywords.csv** — per-language, per-check keyword lists (English and German present) used for substring content detection.
- **recommendations.csv** — semicolon-delimited mapping of industry;category;check_name → business_impact, recommendation, priority (1–10). Used only by the dashboard.
- **benchmark_websites.csv** — URLs flagged as benchmark sites.
- **audit_cache/audits.csv, whois_cache/whois.csv** — the runtime data stores (see §4 for why this design is problematic).

2. Libraries Used

From requirements.txt: **streamlit** (both UIs; note it is listed twice in the file), **pandas** (CSV/DataFrame handling everywhere), **numpy** (averages in benchmarking/statistics), **beautifulsoup4** (all HTML parsing, via html.parser), **requests** (GitHub API commits, raw CSV fetches, crawl fallback), **firecrawl** (primary crawling engine — FirecrawlApp, API key from Streamlit secrets), **python-whois** (domain expiration checks), **plotly** (dashboard charts via plotly.express).

Standard-library usage of note: csv (with field_size_limit(100000000) raised in **both** apps, because the audit CSV stores full page HTML in a single column), concurrent.futures.ThreadPoolExecutor (parallel competitor crawling, MAX_WORKERS = min(32, cpu_count + 4)), base64/json (GitHub Contents API commit payloads), re, urllib.parse, datetime.

Secrets required (Streamlit secrets.toml): GITHUB_REPO, GITHUB_BRANCH, GITHUB_TOKEN, plus the Firecrawl API key. Both apps do st.secrets["GITHUB_REPO"] with direct indexing — a missing key raises KeyError before the friendly error message can be shown (the if not GITHUB_REPO check that follows is dead code for that failure mode).

3. The Two Apps: Roles and Differences

Aspect	Auditor (auditor_app.py)	Dashboard (dashboard.py)
Direction	**Writes** audit data	**Reads** audit data
Input	Single URL or CSV upload	No input; loads published audit CSV
Crawl engine	Firecrawl + requests fallback, ThreadPoolExecutor	None — parses stored HTML only
Scoring	7 audit functions, deductions from score_deductions.csv	None; formats/compares existing scores
GitHub access	Read config CSVs and commit results via Contents API (commit_to_github)	Read-only via raw.githubusercontent.com through github_cache
Cache TTL	github_cache default (2 h)	1 h, with a manual refresh button that calls clear_cache()
Output	On-screen scorecard per URL; persists to audit_cache/audits.csv + whois_cache/whois.csv	Tables, plotly charts, statistics, downloadable CSVs

Auditor flow

Input URL/CSV → normalize/validate → batch (BATCH_SIZE) → per URL: crawl_page() (retries = 2), detect_language(), seven audit functions, get_competitor_benchmarks() → cache → save_cache() commits to GitHub. Cached domains are skipped entirely. complete_missing_audits() retro-fills audit categories for old cache entries missing newer categories. cleanup_old_cache_entries() prunes entries older than CACHE_CLEANUP_DAYS.

Audit categories and what they actually check (all against the single crawled homepage)

- **Technical** — SSL, load time vs threshold, viewport, whois domain expiration, parked-domain detection. Each failed check subtracts its configured deduction.
- **Business info** — keyword-substring checks for products/services, company info, certifications, testimonials; copyright-year regex vs COPYRIGHT_YEAR_THRESHOLD.
- **Functional gaps** — contact form (via CachedSoup.check_contact_form()), RFQ, e-commerce, blog keyword checks.
- **SEO visibility** — meta title (≤60 chars), meta description (50–160 chars), alt text, URL depth, internal links count, Google Maps link/iframe presence, analytics regex.
- **UX** — viewport, base font size, CTAs, form labeling, navigation, heading hierarchy, image alt (hard-coded deductions: 15/10/5, not from score_deductions.csv).
- **Budget red flags** — physical address regex, About/team link, online payments, generic free-email regex, "recent updates" via year regex 202[3-9], DIY-builder keywords.
- **Growth signals** — informational only; always returns score: 100. Detects job postings, LinkedIn widgets, press releases, expansions, GMB, analytics.

Dashboard sections

Latest-audit in-depth analysis; last 10 user-audited sites table; statistical comparison (plotly); industry benchmarks (blending user audits in the same industry); 7-day trends (user audits only); and the key output — "Download Websites by Pain Point" (generate_painpoint_csv_with_contact): a CSV of sites failing a selected check, with contact emails/phones extracted from stored HTML (extract_contact_info). generate_full_csv exports complete audit data. get_recommendation() / get_business_impact() join failed checks to recommendations.csv.

Key coupling risk

The dashboard parses audits.csv rows whose data column is a JSON blob containing full crawl HTML. Any change to the cache schema (domain, data, timestamp) or to the JSON keys (technical, business, functional, seo, ux, budget, growth, benchmarks, crawl, industry_keyword, is_benchmark) breaks the dashboard. There is no schema version field.

4. Known Issues (Evidence-Based)

Critical / architectural

- **(1) GitHub-as-database.** Every audit rewrites the entire audits.csv (including full page HTML) and commits it via the Contents API. The git history (292 commits) is dominated by "Auto-update audits.csv / whois.csv" noise. Problems: the whole file is re-uploaded each save; concurrent audits race on the file sha and silently clobber each other; Contents API rate limits will throttle bulk runs; the repo bloats with every crawl. Recommendation: migrate to any real datastore (SQLite at minimum) before doing anything else.
- **(2) Homepage-only analysis.** Every content check runs on one crawled page, but findings are phrased as site-wide claims ("No blog or resource section detected", "No physical address detected"). Checks that detect links to /about|/team conflate "not on the homepage" with "doesn't exist". This is the single biggest accuracy problem in the scoring.
- **(3) Schema coupling with no versioning** between the two apps (see §3). The dashboard carries a strict expected_columns check for recommendations.csv but the auditor's config loaders use bare except Exception: pass — malformed config CSVs silently fall back to hard-coded defaults with no warning, so the two apps can disagree about configuration without any visible error.

Functional bugs and dead code

- **(4) Unused deductions.** broken_links is configured in score_deductions.csv but no audit function checks links. Same for flash_elements, outdated_plugins, missing_structured_data, missing_security_headers — all present in the default deduction dict, none checked anywhere.
- **(5) technical_audit bypasses the whois cache:** it calls check_domain_expiration(url, {}) with an empty dict, so every technical audit performs a fresh whois lookup (retries per WHOIS_RETRIES) even though a populated whois_cache is available in the calling scope.
- **(6) Silent persistence failure:** commit_to_github returns False and only warns, but save_to_csv/save_cache don't propagate the failure — a batch can "succeed" on screen while writing nothing to the shared store the dashboard reads.
- **(7) Duplicate checks across categories:** viewport (technical + UX), alt text (SEO + UX, different deduction formulas), analytics (SEO + growth), Google Maps (SEO + growth). One defect is reported and penalized multiple times.
- **(8) UX deductions bypass the config:** hard-coded -15/-10/-5 values instead of SCORE_DEDUCTIONS, inconsistent with every other category.

Detection-quality weaknesses

- **(9) Substring keyword matching:** content checks are keyword-in-text against lowercased page text — high false-negative rate (JS-rendered content, synonyms, subpages) and false positives (the word "testimonials" in a menu scores as having testimonials).

- **(10) "Recent updates" check** regexes for year strings 202[3-9] anywhere in text — a copyright line or any date satisfies it.
- **(11) Base font size check** passes if any style block happens to contain "body", "font-size", and one of 16px/1rem/100%/1.0em anywhere — it never verifies an applicable CSS rule.
- **(12) Navigation check** is satisfied by merely finding >5 links starting with / or #.
- **(13) detect_language fallback** counts keyword hits in space-padded lowercase text; crude, and defaults to 'en' on any failure.
- **(14) form_class bug risk in check_contact_form:** form.get("class", "") returns a list when the attribute exists, and .lower() on a list raises AttributeError — any form with a class attribute can crash the contact-form check (it is not wrapped in try/except at the call site).
- **(15) requirements.txt is malformed:** streamlit appears twice; verify the full dependency set with a fresh pip install -r in a clean environment.

Operational notes

- **(16)** st.secrets[...] direct indexing means missing secrets crash rather than show the intended error message (see §2).
- **(17)** github_cache swallows all fetch exceptions and returns an empty DataFrame; downstream code treats empty as "use defaults", so a network outage is indistinguishable from a missing config.
- **(18) Regression history:** a past commit restored four accidentally deleted audit functions (seo_visibility_audit, budget_red_flags_audit, ux_audit, growth_signals_audit) and fixed process_batch. Regression risk around process_batch and the audit-function set is real; there are no tests anywhere in the repo to catch this class of mistake.

5. Suggested Priorities for the Next Developer

- **1.** Replace the GitHub-commit persistence layer with SQLite or any database; keep CSV only as an export format. This also fixes the noise in git history and the concurrency races.
- **2.** Add tests — at minimum, unit tests for each audit function with fixture HTML, and an integration test that a cached schema change doesn't break the dashboard.
- **3.** Crawl beyond the homepage (e.g., homepage + 2–5 linked key pages) before asserting "no blog / no about page / no address".
- **4.** Fix the concrete bugs: check_contact_form class-attribute crash (#14), whois-cache bypass (#5), unused deductions (#4), silent commit failures (#6), secrets KeyError (#16).
- **5.** Deduplicate checks across categories and move UX deductions into score_deductions.csv so scoring weights are configured in one place.
- **6.** Add a schema version field to the cache records so the auditor and dashboard can evolve independently.
- **7.** Clean up requirements.txt and verify the dependency set with a fresh environment install.

The strongest parts of the codebase to preserve: the consistent {score, issues, checks} audit result shape, the externalized CSV configuration, the CachedSoup/github_cache module separation, and the dashboard's pain-point export, which is the product's core value.